Infosys DSE Intensive Workout: Core Patterns

If you want to clear the DSE cutoff, you must be able to solve Easy questions in under 15 minutes and Medium questions in under 30 minutes.

Here are three core patterns to code out today. Try to write the logic yourself before looking at the provided solutions!

Problem 1: The Frequency Map (Easy)

Pattern: Strings & Hash Maps (Dictionaries) **Problem Statement:** Given a string `s`, find the first non-repeating character in it and return its index. If it does not exist, return `-1`. **Why Infosys asks this:** Tests if you know how to use dictionaries for $\mathcal{O}(1)$ time lookups instead of slow nested loops.

Example Input: `infosys` **Example Output:** `1` *(The letter 'n' is the first character that doesn't repeat. 'i' repeats at the end).*

Python Solution Template

```
import sys

def first_unique_char(s):
    # Step 1: Create a dictionary to store character frequencies
    freq = {}
    for char in s:
        freq[char] = freq.get(char, 0) + 1

    # Step 2: Find the first character with a frequency of 1
    for i in range(len(s)):
        if freq[s[i]] == 1:
            return i

    return -1

if __name__ == '__main__':
    # Standard I/O reading
    # input().strip() removes hidden newlines/spaces at the end of the input
    s = input().strip()
    result = first_unique_char(s)
    print(result)
```

Problem 2: The Two-Pointer Swap (Medium)

Pattern: Two Pointers in an Array **Problem Statement:** Given an integer array `nums`, move all `0`s to the end of it while maintaining the relative order of the non-zero elements. You must do this in-place without making a copy of the array. **Why Infosys asks this:** Tests your ability to manipulate array indices in $\mathcal{O}(N)$ time. Brute-forcing this will cause a Time Limit Exceeded (TLE) error.

Example Input: `0 1 0 3 12` **Example Output:** `1 3 12 0 0`

Python Solution Template

```
def move_zeroes(nums):
    # 'left' pointer keeps track of where the next non-zero should go
    left = 0

    # 'right' pointer scans through the array
    for right in range(len(nums)):
        if nums[right] != 0:
            # Swap the elements
            nums[left], nums[right] = nums[right], nums[left]
            left += 1

    return nums

if __name__ == '__main__':
    # Reading a space-separated string of numbers into a list of integers
    nums = list(map(int, input().split()))

    result = move_zeroes(nums)

    # Printing the list as a space-separated string
    print(*(result))
```

Problem 3: Kadane's Algorithm Refresher (Medium/DSE Level)

Pattern: Dynamic Programming (1D) / Greedy **Problem Statement:** Given an integer array `nums`, find the contiguous subarray (containing at least one number) which has the largest sum and return its sum. **Why Infosys asks this:** This is a legendary interview question. If you write a nested `for` loop, your complexity is $\mathcal{O}(N^2)$ and you will fail the hidden test cases. You must do it in $\mathcal{O}(N)$.

Example Input: `-2 1 -3 4 -1 2 1 -5 4` **Example Output:** `6` *(The subarray is* `4 -1 2 1` *)*

Python Solution Template

```
def max_subarray(nums):
    current_sum = nums[0]
    max_sum = nums[0]

    for i in range(1, len(nums)):
        # At each step, do we start a new subarray or add to the current one?
        current_sum = max(nums[i], current_sum + nums[i])

        # Update the overall maximum found so far
        max_sum = max(max_sum, current_sum)

    return max_sum

if __name__ == '__main__':
    nums = list(map(int, input().split()))
    result = max_subarray(nums)
    print(result)
```